

Chương 5

Người dùng & phân quyền

ai được làm gì (chmod, sudo)

Nối tiếp Chương 1-4: bạn đã biết `cd`, `ls`, `file`, chuyển hướng (`>`), đường ống (`|`), và đã từng nhìn thấy cột quyền lạ lạ kiểu `-rw-r--r--` khi gõ `ls -l`. Chương này sẽ giải mã đúng cái cột đó, và dạy bạn cách thay đổi “ai được làm gì” với file của mình.

Tài liệu học Linux / DevOps cho người mới

Thực hành trên macOS (shell `zsh`)

Mục lục

1	Mục tiêu học tập	2
2	Vì sao cần phân quyền? (câu chuyện đa người dùng)	2
3	Thực hành ngay (lab A): Tôi là ai trên máy này?	3
4	Đọc cột quyền trong ls -l (giải mã 10 ký tự)	4
4.1	Ký tự đầu tiên: loại đối tượng	4
4.2	Ba cụm rwx: quyền cho từng vai	5
4.3	x trên FILE khác x trên THƯ MỤC — đừng nhầm!	5
5	Thực hành ngay (lab B): Quyền x “sống động” — chạy một script	6
6	chmod — hai cách đổi quyền	8
6.1	Cách KÝ HIỆU (symbolic) — dễ đọc cho người	8
6.2	Cách SỐ (numeric / octal) — gọn cho dân quen	9
7	Thực hành ngay (lab C): File riêng tư + so sánh hai cách	10
8	chown / chgrp — đổi CHỦ và đổi NHÓM (sơ lược)	12
9	sudo — mượn quyền quản trị trong chốc lát	13
9.1	sudo là gì?	13
9.2	sudo khác su thế nào?	13
9.3	“Quyền lớn, trách nhiệm lớn” — cảnh báo cho người mới	14
10	Góc Senior (vì sao, production, cạm bẫy)	14
11	Hiểu lầm thường gặp (đọc kỹ để khỏi sai cả đời)	15
12	Thẻ ghi nhớ (flashcards)	16
13	Kế hoạch ôn ngắt quãng	17
14	Tóm tắt 1 trang	17
15	Bài tập mở rộng (có đáp án)	18
	Đáp án các mục “Tự kiểm tra”	19

1 Mục tiêu học tập

Học xong chương này, bạn sẽ:

1. Hiểu vì sao Linux/Unix (và macOS) là hệ **đa người dùng**, và vì sao điều đó bắt buộc phải có **phân quyền**.
2. Biết mình là **ai** trên máy: dùng `whoami`, `id`, `groups`.
3. **Đọc trôi chảy** chuỗi 10 ký tự quyền trong `ls -l` (ví dụ `-rwxr-xr--`), phân biệt quyền của **chủ sở hữu (owner)** / **nhóm (group)** / **người khác (others)**.
4. Hiểu ý nghĩa **r (đọc)**, **w (ghi)**, **x (thực thi)** — và đặc biệt vì sao **x** trên **file** khác **x** trên **thư mục**.
5. Dùng `chmod` thành thạo theo **2 cách**: ký hiệu (`u+x`) và số (`755`, `644`, `600`).
6. Biết sơ về `chown/chgrp` (đổi chủ/nhóm) và sự khác biệt trên macOS.
7. Hiểu `sudo` là gì, khi nào cần, vì sao **nguy hiểm nếu lạm dụng**, và khác su ra sao.
8. Nắm tư duy **least privilege (đặc quyền tối thiểu)** — nền tảng bảo mật của mọi DevOps/Senior.

Trấn an

Toàn bộ phần thực hành ở chương này **không dùng sudo, không đụng file hệ thống**. Bạn chỉ nghịch trong sân tập `~/linux-lab`. An toàn tuyệt đối, có lỗi gõ sai cũng không hỏng máy.

2 Vì sao cần phân quyền? (câu chuyện đa người dùng)

Hãy tưởng tượng một **văn phòng dùng chung**: nhiều người ngồi chung, mỗi người có một ngăn tủ khóa riêng. Bạn không muốn đồng nghiệp mở tủ bạn lấy đồ, và sếp (người có chìa khóa tổng) thì mở được mọi tủ khi cần.

Unix/Linux sinh ra trong môi trường y hệt: **một máy chủ, nhiều người dùng** đăng nhập cùng lúc, chạy nhiều tiến trình (chương trình đang chạy) song song. Để mọi người không giẫm chân/đọc trộm/xóa nhầm của nhau, hệ điều hành gán cho **mỗi file và thư mục** một bộ quy tắc: *ai được đọc, ai được sửa, ai được chạy*.

macOS kế thừa nguyên vẹn triết lý này (vì macOS có lỗi Unix BSD). Dù máy bạn chỉ có một mình bạn dùng, hệ điều hành vẫn âm thầm chạy hàng chục **tài khoản hệ thống** (cho tiến trình nền) — nên phân quyền vẫn hoạt động đầy đủ.

Ba “vai” cần nhớ với mỗi file:

Vai	Tiếng Anh	Là ai
Chủ sở hữu	owner (hay <i>user</i>)	Người tạo/sở hữu file
Nhóm	group	Một nhóm tài khoản; file thuộc về 1 nhóm
Người khác	others	Tất cả những ai còn lại

Và trên đỉnh kim tự tháp là **root** (còn gọi **superuser** — siêu người dùng): tài khoản có **quyền tối cao**, làm được mọi thứ, bỏ qua mọi rào quyền. (Ta sẽ gặp root khi nói về `sudo` ở mục 9.)

Định nghĩa nhanh

- **uid** = User ID = số định danh người dùng (vd root luôn là uid 0).
- **gid** = Group ID = số định danh nhóm.
- **tiền trình (process)** = một chương trình đang chạy.

Tự kiểm tra 5.1

1. Ba “vai” quyền của một file là gì?
2. root có gì đặc biệt?
3. Vì sao máy cá nhân (chỉ mình bạn) vẫn cần phân quyền?

3 Thực hành ngay (lab A): Tôi là ai trên máy này?

Mở **Terminal** (ứng dụng Terminal trên macOS, shell **zsh**). Gõ từng dòng, nhấn Enter sau mỗi dòng.

Lab A — Nhận diện danh tính

```
whoami
```

Kết quả mong đợi: in ra tên đăng nhập của bạn, ví dụ:

```
vinh
```

```
id
```

Kết quả mong đợi: một dòng dài chứa uid, gid và danh sách nhóm, ví dụ:

```
uid=501(vinh) gid=20(staff) groups=20(staff),12(everyone),61(localaccounts),...
```

- **uid=501(vinh)**: bạn là người dùng số 501, tên vinh. (Trên macOS, tài khoản người đầu tiên thường bắt đầu từ **501**.)
- **gid=20(staff)**: nhóm chính (primary group) của phiên làm việc hiện tại là **staff** (số 20). Đây là điểm **đặc trưng của macOS** — trên Linux, nhóm chính thường trùng tên người dùng.
- **groups=...**: tất cả các nhóm bạn thuộc về.

```
groups
```

Kết quả mong đợi: liệt kê tên các nhóm, ví dụ:

```
staff everyone localaccounts ...
```

Máy bạn có thể khác

Tên đăng nhập, số uid, và danh sách nhóm sẽ khác — đó là bình thường. Quan trọng là bạn thấy được **3 lệnh này trả lời câu “tôi là ai”** như thế nào. Nếu bạn là tài khoản đầu tiên trên Mac, gần như chắc chắn nhóm chính là **staff**.

Góc thuật ngữ

whoami = “who am I” (tôi là ai). **id** = identity (danh tính). Dễ nhớ.

Tự kiểm tra 5.2

1. Lệnh nào cho biết **tên đăng nhập** của bạn?
2. Trong **id**, **uid** và **gid** khác nhau ở chỗ nào?
3. Nhóm chính mặc định trên macOS thường tên là gì?

4 Đọc cột quyền trong `ls -l` (giải mã 10 ký tự)

Đây là kỹ năng **quan trọng nhất** của chương. Khi gõ `ls -l`, mỗi dòng bắt đầu bằng một chuỗi **10 ký tự**, ví dụ:

```
-rwxr-xr--
```

Ta chia nó thành **1 + 3 + 3 + 3**:

-	rwx	r-x	r--
^	^	^	^
loai	owner	group	others
file	(chu)	(nhom)	(nguoi khac)

4.1 Ký tự đầu tiên: loại đối tượng

Ký tự	Nghĩa
-	file thường (regular file)
d	thư mục (d irectory)
l	liên kết tượng trưng (link — như “shortcut”)

Còn vài loại khác ít gặp

c và **b** (thiết bị — character/block device), **p** (named pipe/FIFO), **s** (socket). Khi mới học bạn gần như không gặp; chỉ cần biết tên để không bất ngờ khi sau này thấy chúng ở các thư mục hệ thống.

Đặc trưng macOS — dấu @ và +

Trên macOS bạn có thể thấy thêm một dấu ngay **sau** 10 ký tự, ví dụ **-rw-r--r--@**. Dấu @ nghĩa là file có **thuộc tính mở rộng (extended attributes)** — KHÔNG phải quyền r/w/x, cứ bỏ qua khi mới học (muốn xem chi tiết: `xattr -l tenfile`). Dấu + (nếu thấy) nghĩa là file có **ACL** (danh sách quyền chi tiết hơn). Cả hai đều không ảnh hưởng tới bài học về r/w/x.

4.2 Ba cụm rwx: quyền cho từng vai

Mỗi cụm có **3 vị trí cố định**: r rồi w rồi x. Nếu có quyền thì hiện chữ, không có thì hiện dấu -.

Chữ	Tên	Giá trị số
r	read — đọc	4
w	write — ghi/sửa	2
x	execute — thực thi	1
-	không có quyền đó	0

Ví dụ giải mã **-rwxr-xr--**:

Cụm	Ký tự	Nghĩa
Loại	-	file thường
owner	rwx	chủ đọc + ghi + chạy được
group	r-x	nhóm đọc + chạy được, KHÔNG sửa được
others	r--	người khác CHỈ đọc được

4.3 x trên FILE khác x trên THƯ MỤC — đừng nhầm!

Đây là chỗ người mới hay rối:

- **x trên một FILE** = “file này **chạy được** như một chương trình/script”. Không có x thì dù file là script đúng, hệ thống vẫn **từ chối chạy**.
- **x trên một THƯ MỤC** = “được phép **đi vào** (cd) thư mục đó và truy cập file bên trong”. Một thư mục có r nhưng không có x thì bạn **liệt kê được tên file nhưng không vào được**.

Ghi nhớ một câu

Với thư mục, x nghĩa là “*được bước qua cửa*”; với file, x nghĩa là “*được khởi động nó*”.

Tự kiểm tra 5.3

1. Trong **-rw-r--r--**, **owner** có những quyền nào? **others** có quyền gì?
2. **r=?**, **w=?**, **x=?** (giá trị số).
3. Một thư mục có quyền r nhưng thiếu x thì bạn làm được gì và không làm được gì?

gì?

5 Thực hành ngay (lab B): Quyền x “sống động” — chạy một script

Giờ ta sẽ tận mắt thấy quyền x quyết định “chạy được hay không”. **Gõ tay** từng dòng.

Lab B — Bước 1: Tạo sân tập

```
mkdir -p ~/linux-lab/ch5
cd ~/linux-lab/ch5
```

- `mkdir -p`: tạo thư mục (cờ `-p` = tạo cả thư mục cha nếu chưa có, và không báo lỗi nếu đã tồn tại).
- `cd`: vào thư mục đó.

Kết quả mong đợi: không in gì ra cả (im lặng = thành công). Kiểm chứng bằng:

```
pwd
```

in ra đường dẫn, ví dụ `/Users/vinh/linux-lab/ch5`.

Lab B — Bước 2: Tạo một script nhỏ

```
echo 'echo "xin chào tu script"' > chao.sh
ls -l chao.sh
```

Kết quả mong đợi (trên macOS, chú ý dấu @):

```
-rw-r--r--@ 1 vinh  staff  26 Jun 26 10:00 chao.sh
```

Đọc dòng này: cột đầu là **quyền**; sau quyền là **số liên kết** (1); rồi **tên chủ sở hữu** (vinh); rồi **nhóm của file** (staff); rồi **số byte**, ngày giờ, và tên file.

Nhìn cụm quyền: `-rw-r--r--`. Owner đọc+ghi, group đọc, others đọc. **Không hề có x ở đâu cả** — nghĩa là file này chưa “chạy được”.

Máy bạn có thể khác — đọc kỹ để khỏi hoang mang

- **Dấu @**: Trên macOS hiện đại, file tạo bằng `echo ... > file` gần như luôn nhận một thuộc tính mở rộng (vd `com.apple.provenance`), nên `ls -l` hiển thị dấu @ ngay sau cụm quyền (`-rw-r--r--@`). Đây là **bình thường**, dấu @ KHÔNG phải quyền và không ảnh hưởng gì tới r/w/x. Nếu máy bạn không có @ cũng không sao.
- **Cột group (staff hay wheel?)**: Cột thứ tư có thể hiển thị staff HOẶC

wheel — tùy thư mục bạn đang đứng. macOS (lỗi BSD) cho file mới **kế thừa nhóm của thư mục cha**, nên ở `~/linux-lab` thường là `staff`, còn ở vài nơi khác (vd `/tmp`) lại là `wheel`. Cả hai đều bình thường, không ảnh hưởng bài học.

- **Số byte** (vd 26), **ngày giờ**, **tên đăng nhập** đương nhiên khác. Phần cụm quyền `-rw-r--r--` thì gần như chắc chắn giống — đó là quyền mặc định cho file mới tạo.

Lab B — Bước 3: Thử chạy khi CHƯA có x

```
./chao.sh
```

Kết quả mong đợi: bị từ chối, báo đại loại:

```
zsh: permission denied: ./chao.sh
```

Lưu ý máy bạn có thể khác: phần tiền tố trước có thể là `zsh:` hoặc `(eval):1:` tùy cấu hình shell — không sao. Nếu bạn lỡ dùng `bash` thay vì `zsh`, thông báo sẽ hơi khác: `bash: ./chao.sh: Permission denied` — ý nghĩa giống hệt. Đây chính là minh họa **sống động**: file nội dung đúng, nhưng thiếu quyền `x` nên hệ thống **không cho chạy**. (`./` nghĩa là “file nằm ngay trong thư mục hiện tại”.)

Lab B — Bước 4: Thêm quyền x cho owner, rồi chạy lại

```
chmod u+x chao.sh
ls -l chao.sh
```

Kết quả mong đợi: giờ có `x` ở cụm owner:

```
-rwxr--r--@ 1 vinh staff 26 Jun 26 10:00 chao.sh
```

Chạy lại:

```
./chao.sh
```

Kết quả mong đợi:

```
xin chao tu script
```

Bạn vừa chứng kiến: **chỉ cần thêm một chữ x**, file từ “không chạy được” thành “chạy được”. Đó là sức mạnh của phân quyền.

Giải thích `chmod u+x`

`chmod` = *change mode* (đổi quyền). `u` = user/owner. `+x` = thêm quyền execute. Đọc liền: “thêm quyền chạy cho chủ sở hữu”.

Vì sao script này chạy được

Script chạy được là nhờ shell hiện tại (**zsh**) tự diễn giải nội dung. Ở chương sau bạn sẽ học thêm dòng đầu **#!/bin/sh** (gọi là **shebang**) để chỉ định rõ trình thông dịch cho script — khi đó file có thể chạy độc lập, không phụ thuộc shell đang dùng.

Tự kiểm tra 5.4

1. Vì sao lần đầu `./chao.sh` báo `permission denied`?
2. `chmod u+x chao.sh` làm gì?
3. `./` ở đầu lệnh nghĩa là gì?

6 chmod — hai cách đổi quyền

6.1 Cách KÝ HIỆU (symbolic) — dễ đọc cho người

Cú pháp: `chmod [ai] [+/-/=] [quyền] file`

Phần “ai”:

Ký tự	Nghĩa
u	user/owner (chủ)
g	group (nhóm)
o	others (người khác)
a	all (cả ba — all)

Phần “phép toán”:

Dấu	Nghĩa
+	thêm quyền (cộng dồn vào quyền hiện có)
-	bớt quyền
=	đặt đúng bằng (xóa các quyền khác không liệt kê)

Phân biệt quan trọng + và =

+x chỉ **thêm** quyền x, KHÔNG đụng tới r/w đang có. Còn **=** **đặt tuyệt đối**: **g=r** nghĩa là group chỉ còn đúng r, mọi quyền khác của group (kể cả w nếu đang có) bị xóa. Khi muốn **chắc chắn khóa** một quyền của ai đó, dùng **=** (hoặc cách số), đừng dùng **+**.

Ví dụ:

```
chmod u+x file      # thêm thực thi cho chủ (không đụng quyền khác)
chmod go-w file     # bỏ quyền ghi của group VÀ others
chmod a+r file      # cho tất cả đọc được
chmod o= file       # xóa sạch mọi quyền của others
```

6.2 Cách SỐ (numeric / octal) — gọn cho dân quen

Thuật ngữ

octal = hệ **bát phân**, mỗi chữ số chạy từ **0 đến 7**. Hệ này vừa khít với quyền vì mỗi vai chỉ có 3 quyền **r/w/x** (tối đa $4+2+1 = 7$).

Mỗi vai là một **chữ số** = tổng của $r(4) + w(2) + x(1)$:

Số	Quyền	Nghĩa
7	rwX	đọc + ghi + chạy ($4+2+1$)
6	rw-	đọc + ghi ($4+2$)
5	r-X	đọc + chạy ($4+1$)
4	r--	chỉ đọc
0	---	không có gì

Một lệnh **chmod** số gồm **3 chữ số**: owner, group, others — theo đúng thứ tự.
Ba con số “quốc dân” phải thuộc lòng:

Lệnh	Quyền	Dùng cho
chmod 755	rwXr-Xr-X	script & thư mục cần cho người khác truy cập : chủ làm tất, người khác đọc/chạy/vào được
chmod 644	rw-r--r--	file thường : chủ sửa, người khác chỉ đọc
chmod 600	rw-----	file riêng tư (vd khóa SSH): chỉ mình chủ đọc/ghi

Mẹo nhớ

755 = “7 cho mình, 5-5 cho thiên hạ”. 644 = “6 cho mình, 4-4 cho thiên hạ”. 600 = “6 cho mình, 0-0 cho thiên hạ” (khóa kín).

Đừng “cứ 755 cho mọi thứ”

755 hợp với script và thư mục **cần** cho người khác vào. Nhưng với thư mục/dữ liệu **riêng tư**, theo tư duy least privilege (mục 10) bạn nên cân nhắc 700 (chỉ mình bạn), 750 (mình + nhóm đọc/vào), hay 600 cho file riêng. Cấp đúng mức tối thiểu cần dùng, không cấp dư.

Tự kiểm tra 5.5

1. `chmod go-w abc` làm gì?
2. Số 5 tương ứng quyền nào? 6? 7?
3. `chmod 600` cho ra chuỗi quyền nào, và phù hợp với loại file gì?

7 Thực hành ngay (lab C): File riêng tư + so sánh hai cách

Vẫn đang ở ~/linux-lab/ch5. (Nếu lỡ thoát, gõ lại `cd ~/linux-lab/ch5.`)

Lab C — Bước 1: File riêng tư với 600

```
echo "rieng tu" > bi-mat.txt
chmod 600 bi-mat.txt
ls -l bi-mat.txt
```

Kết quả mong đợi:

```
-rw-----@ 1 vinh staff 9 Jun 26 10:05 bi-mat.txt
```

Chỉ owner đọc/ghi; group và others **trống trơn** (-----). Đây đúng là kiểu quyền dùng cho khóa bí mật. (Dấu @ nếu có chỉ là thuộc tính mở rộng của macOS, bỏ qua.)

Lab C — Bước 2: Quan sát go-w có TÁC DỤNG THẬT

Để thấy rõ go-w *gỡ* quyền ghi, ta cố tình tạo trạng thái có w cho group và others trước:

```
chmod 664 bi-mat.txt
ls -l bi-mat.txt
```

Kết quả mong đợi: group và others đều có w:

```
-rw-rw-r--@ 1 vinh staff 9 Jun 26 10:05 bi-mat.txt
```

Giờ gỡ w của group + others:

```
chmod go-w bi-mat.txt
ls -l bi-mat.txt
```

Kết quả mong đợi: w của group biến mất:

```
-rw-r--r--@ 1 vinh staff 9 Jun 26 10:05 bi-mat.txt
```

Quan sát: lần này bạn **thấy rõ** w bị gỡ (từ `rw-rw-` về `rw-r--`). Nếu chạy `go-w` trên một file vốn đã không có w cho group/others (vd 644), lệnh vẫn chạy đúng nhưng **không có gì thay đổi nhìn thấy được** — đừng tưởng lệnh hỏng. Đặt lại 600 cho an toàn rồi sang bước sau:

```
chmod 600 bi-mat.txt
```

Lab C — Bước 3: So sánh số 644 và 755 trên chao.sh

```
chmod 644 chao.sh
ls -l chao.sh
```

Kết quả mong đợi: mất x, quay về file thường:

```
-rw-r--r--@ 1 vinh staff 26 Jun 26 10:00 chao.sh
```

```
chmod 755 chao.sh
ls -l chao.sh
```

Kết quả mong đợi: ai cũng chạy/đọc được, chỉ chủ sửa được:

```
-rwxr-xr-x@ 1 vinh staff 26 Jun 26 10:00 chao.sh
```

Quan sát: cùng một file, 644 và 755 chỉ khác nhau ở các chữ x. Bạn vừa “lập trình quyền” bằng con số. So với `chmod u+x` ở lab B — hai cách **cho kết quả tương đương**, chỉ là cách viết khác nhau.

Lab C — Bước 4: Đối chiếu lại bằng ký hiệu (tùy chọn)

```
chmod go-x chao.sh
ls -l chao.sh
```

Kết quả mong đợi: bỏ x của group+others, còn lại owner chạy được:

```
-rwxr--r--@ 1 vinh staff 26 Jun 26 10:00 chao.sh
```

Lab C — Bước 5: DỌN DẸP (quan trọng)

```
cd ~
rm -r ~/linux-lab/ch5
```

- `cd ~`: về thư mục nhà (~ = home của bạn).
- `rm -r`: xóa thư mục cùng mọi thứ bên trong (-r = recursive, đệ quy).

Kết quả mong đợi: im lặng = đã xóa sạch sân tập ch5. Kiểm chứng:

```
ls ~/linux-lab
```

sẽ không còn thấy ch5.

Trần an + cảnh báo

`rm -r` ở đây chỉ xóa đúng thư mục tập của bạn (`~/linux-lab/ch5`), không liên quan gì tới hệ thống — an toàn. NHƯNG hãy ghi nhớ nguyên tắc chung: **tuyệt đối không** gõ `rm -r` (nhất là kèm `sudo`) với đường dẫn bắt đầu bằng `/` hay `~` trừ khi bạn chắc 100%. Luôn `ls` đường dẫn để nhìn rõ mình sắp xóa gì **trước khi** xóa.

Tự kiểm tra 5.6

1. Sau `chmod 600 bi-mat.txt`, group và others thấy được gì?
2. Khác biệt hiển thị giữa 644 và 755 nằm ở ký tự nào?
3. `rm -r` khác `rm` thường ở chỗ nào?

8 `chown` / `chgrp` — đổi CHỦ và đổi NHÓM (sơ lược)

Khác với `chmod` (đổi *quyền*), hai lệnh này đổi *ai sở hữu*:

- **`chown`** = *change owner* — đổi chủ sở hữu file.
Ví dụ: `chown vinh tepnao.txt` (đổi chủ thành vinh).
Đổi cả chủ và nhóm: `chown vinh:staff tepnao.txt`.
- **`chgrp`** = *change group* — đổi nhóm của file.
Ví dụ: `chgrp staff tepnao.txt`.

Các lệnh trên chỉ để NHẬN DIỆN — đừng chạy trong lab

Đổi chủ sở hữu file sang **người dùng khác LUÔN** cần quyền root (`sudo`) trên cả Linux và macOS — người dùng thường không thể tùy ý “cho/nhận” chủ sở hữu (nếu cho phép thì hết bảo mật). Nếu bạn thử `chown` sang user khác mà không có quyền, bạn sẽ gặp lỗi `Operation not permitted` — **đừng** vì thế mà gõ `sudo`; bạn không cần làm điều này khi mới học.

Riêng **`chgrp`**: người dùng thường có thể đổi nhóm file của mình sang một nhóm **mà chính họ là thành viên** mà không cần `sudo`; đổi sang nhóm khác thì cần root.

Khác biệt trên macOS

macOS dùng nhóm `staff` làm nhóm chính (Linux thường tạo nhóm riêng theo tên user). Cú pháp `chown user:group` thì giống nhau. macOS còn có lớp bảo vệ riêng (vd các thư mục hệ thống bị khóa bởi cơ chế bảo vệ toàn vẹn) nên kể cả `sudo` cũng có chỗ không đổi được.

Phân biệt cốt lõi — đừng nhầm:

Lệnh	Đổi cái gì
<code>chmod</code>	Quyền (ai được đọc/ghi/chạy)
<code>chown</code>	Chủ sở hữu (file thuộc về ai)
<code>chgrp</code>	Nhóm của file

9 sudo — mượn quyền quản trị trong chốc lát

9.1 sudo là gì?

sudo = *superuser do* = “làm với tư cách siêu người dùng”. Bạn đặt **sudo** trước một lệnh để **chạy đúng lệnh đó** với quyền quản trị (quyền của root), thường để:

- sửa **file hệ thống** (file của hệ điều hành),
- **cài đặt** phần mềm vào khu vực chung của máy,
- khởi động/dừng dịch vụ hệ thống.

Cú pháp:

```
sudo <lệnh can quyền cao>
```

Lần đầu trong phiên, **sudo** sẽ **hỏi mật khẩu**. Trên **macOS**, đó chính là **mật khẩu đăng nhập của CHÍNH bạn** (với điều kiện tài khoản của bạn thuộc nhóm admin/sudoers). Khi gõ, màn hình **không hiện ký tự nào** (kể cả dấu sao); cứ gõ rồi Enter. Sau khi đúng, macOS thường nhớ trong vài phút để bạn không phải gõ lại liên tục.

CẢNH BÁO bảo mật — đọc kỹ

Chỉ gõ mật khẩu khi **CHÍNH BẠN** vừa chủ động gõ một lệnh bắt đầu bằng **sudo**. Nếu một script lạ, một hướng dẫn trên mạng, hay bất kỳ thứ gì **tự động** hiện ô hỏi mật khẩu **sudo**, **ĐỪNG** gõ — hãy dừng lại và kiểm tra xem lệnh đó làm gì. Đây là vector lừa đảo (phishing) phổ biến với người mới. (Liên hệ với cảnh báo copy-paste ở mục 9.3.)

Trên macOS

Tài khoản người dùng đầu tiên thường là **admin** và có quyền dùng **sudo**. Tài khoản “standard” (chuẩn) thì không. Bạn **không cần** thử **sudo** trong chương này.

9.2 sudo khác su thế nào?

- **su** (*switch user*) = **chuyển hẳn** sang một người dùng khác (mặc định là root) và **ở lại đó** cho tới khi **exit**. Giống như “ngồi vào ghế root rồi làm liên tục”.
- **sudo** = chỉ **mượn quyền cho đúng một lệnh** rồi trả lại ngay. Giống như “nhờ sếp ký duyệt một giấy tờ” thay vì “chiếm ghế sếp”.

sudo an toàn hơn vì **phạm vi hẹp** (một lệnh) và **có ghi log** (xem mục Senior). Trên macOS, việc **đăng nhập trực tiếp bằng tài khoản root bị vô hiệu hóa mặc định** (root không có mật khẩu để đăng nhập trực tiếp), NHƯNG root vẫn tồn tại và **sudo** vẫn nâng quyền lên root (uid 0) để chạy lệnh — đó chính là lý do **sudo** là con đường chuẩn. Nói cách khác: macOS không cho bạn “đăng nhập làm root”, nhưng **sudo** vẫn chạy lệnh **với tư cách root**.

9.3 “Quyền lớn, trách nhiệm lớn” — cảnh báo cho người mới

`sudo` bỏ qua mọi rào quyền. Một lệnh xóa nhầm chạy với `sudo` có thể **phá hệ điều hành**. Nguyên tắc vàng:

Nguyên tắc vàng khi dùng `sudo`

- **KHÔNG** gõ `sudo` chỉ vì thấy lệnh báo lỗi quyền — hãy hiểu *vì sao* nó cần quyền cao trước đã.
- **KHÔNG** copy-paste lệnh `sudo` lạ từ Internet khi chưa hiểu nó làm gì.
- **Đặc biệt nguy hiểm:** tuyệt đối tránh tổ hợp `sudo rm -r` với đường dẫn bắt đầu bằng `/` hoặc `~` (vd `rm -rf /` / có thể xóa sạch máy). Luôn `ls` đường dẫn trước khi xóa.
- Khi học, gần như **mọi việc luyện tập đều KHÔNG cần `sudo`** (như cả chương này).

Tự kiểm tra 5.7

1. `sudo` viết tắt của gì? Dùng để làm gì?
2. Trên macOS, `sudo` hỏi mật khẩu nào? Vì sao gõ không thấy ký tự?
3. `sudo` khác su ở điểm cốt lõi nào?

10 Góc Senior (vì sao, production, cạm bẫy)

1. Phân quyền là XƯƠNG SỐNG của bảo mật

Cả ngành DevOps/Security xoay quanh một câu hỏi: “*Thực thể này có thực sự cần quyền đó không?*” Nguyên tắc nền là **least privilege (đặc quyền tối thiểu)**: cấp **đúng mức tối thiểu** đủ để làm việc, không hơn. Một tài khoản/tiến trình bị chiếm cũng chỉ gây hại trong phạm vi quyền nó có.

2. Vì sao `chmod 777` là NGUY HIỂM

`777 = rwxrwxrwx = ai cũng đọc/ghi/chạy được`. Nghĩa là bất kỳ người dùng hay tiến trình nào trên máy đều có thể **sửa hoặc thay thế** file đó — kẻ tấn công có thể thay script của bạn bằng mã độc. Câu “cứ 777 cho chắc, đỡ lỗi quyền” là **sai lầm kinh điển** che giấu lỗ hổng thật. Cực kỳ tránh `chmod -R 777` (đệ quy cả cây thư mục) — đây là một trong những thói quen tệ nhất, mở toang toàn bộ dự án.

3. Ví dụ thật: khóa SSH bắt buộc 600

Khóa riêng SSH (private key, dùng để đăng nhập server từ xa) **buộc** phải có quyền 600 (chỉ chủ đọc/ghi). Nếu lỏng hơn (vd group/others đọc được), chương trình `ssh`

sẽ **từ chối dùng khóa** và báo lỗi đại loại “*permissions are too open*”. Đây là ví dụ sống động: phần mềm chủ động **ép** bạn theo least privilege.

4. Bộ ba nâng cao (biết tên để sau này tra cứu)

- **setuid / setgid**: cho phép một chương trình chạy với quyền của *chủ file* thay vì người gọi (mạnh nhưng nhạy cảm về bảo mật).
- **sticky bit**: thường đặt trên thư mục dùng chung (như `/tmp`) để **chỉ chủ file mới xóa được file của mình**, dù thư mục mở cho nhiều người ghi.

(Chương này chỉ giới thiệu tên; bạn sẽ học sâu ở chương nâng cao.)

5. sudo có AUDIT (ghi log)

Mỗi lần **sudo**, hệ thống ghi lại *ai, lúc nào, chạy lệnh gì*. Trong production, đây là cơ sở để **truy vết** khi có sự cố — và là lý do **sudo** được ưa hơn việc “thành root rồi làm gì cũng được mà không dấu vết”.

6. Liên hệ tương lai (cloud/IAM)

Mô hình owner/group/others + least privilege chính là **phiên bản sơ khai** của **IAM** (Identity and Access Management) trên cloud (AWS/GCP/Azure): ở đó bạn sẽ gán “policy” cho “user/role” theo đúng tư duy *ai được làm gì với tài nguyên nào*. Học chắc **chmod/sudo** hôm nay là đặt nền cho IAM ngày mai.

11 Hiểu lầm thường gặp (đọc kỹ để khỏi sai cả đời)

Hiểu lầm	Sự thật
“Cứ chmod 777 cho chắc, đỡ lỗi quyền.”	SAI . 777 mở toang cho mọi người ghi → lỗ hổng bảo mật. Đặt đúng quyền tối thiểu cần dùng (vd 644, 755, 600).
“root và sudo là một.”	Không . root là tài khoản quyền tối cao. sudo là công cụ để mượn quyền root cho một lệnh rồi trả lại. Bạn không “là” root khi dùng sudo.
“Đổi quyền = đổi chủ sở hữu.”	Khác nhau . chmod đổi quyền (đọc/ghi/chạy). chown đổi chủ . Hai việc hoàn toàn riêng.
“Thiếu x thì cứ sửa nội dung file là chạy được.”	Không . Nội dung đúng nhưng thiếu bit x thì hệ thống vẫn từ chối chạy (đã thấy ở lab B).
“Thư mục có r là vào được rồi.”	Không . Vào (cd) cần x. r chỉ cho liệt kê tên .

Hiểu lầm	Sự thật
“+x cũng khóa được quyền ghi của người khác.”	Không. +x chỉ thêm x, không đụng w. Muốn khóa quyền ghi của group/others phải dùng = (đặt tuyệt đối) hoặc cách số.
“macOS không có root.”	Không. root vẫn tồn tại; chỉ là đăng nhập trực tiếp bằng root bị tắt mặc định. sudo vẫn chạy lệnh với tư cách root.

12 Thẻ ghi nhớ (flashcards)

Cách dùng

Che cột phải, tự trả lời, rồi mở ra kiểm tra.

Hỏi	Đáp
whoami làm gì?	In tên đăng nhập của tôi
id cho biết gì?	uid, gid (nhóm chính của phiên), các nhóm
Ký tự đầu d trong ls -l nghĩa là?	Thư mục (directory)
Dấu @ sau cụm quyền trên macOS?	File có thuộc tính mở rộng (extended attributes), không phải quyền
r, w, x giá trị số?	4, 2, 1
7 = quyền gì?	rw- (đọc+ghi+chạy)
5 = quyền gì?	r-x (đọc+chạy)
chmod 644 → chuỗi?	rw-r--r-- (file thường)
chmod 755 → chuỗi?	rw-r-xr-x (script & thư mục cần cho người khác truy cập; thư mục riêng tư nên xét 700/750)
chmod 600 → chuỗi?	rw----- (file riêng tư/khóa)
chmod u+x file	Thêm quyền chạy cho chủ (không đụng quyền khác)
chmod go-w file	Bỏ quyền ghi của group+others
+x vs =?	+x thêm x (cộng dồn); = đặt tuyệt đối, xóa quyền không liệt kê
x trên thư mục nghĩa là?	Được cd vào (bước qua cửa)
chmod vs chown?	Đổi quyền vs đổi chủ sở hữu
sudo là gì?	Chạy 1 lệnh với quyền quản trị (superuser do)
sudo vs su?	Mượn quyền 1 lệnh vs chuyển hẳn sang user khác
macOS có root không?	Có; chỉ tắt đăng nhập trực tiếp, sudo vẫn chạy lệnh với tư cách root
Vì sao khóa SSH cần 600?	Lỏng hơn thì ssh từ chối dùng
Vì sao 777 nguy hiểm?	Ai cũng ghi được → lỗ hổng bảo mật

Hỏi	Đáp
Least privilege nghĩa là?	Cấp đúng quyền tối thiểu đủ dùng

13 Kế hoạch ôn ngắt quãng

- **Sau 1 ngày:** Mở Terminal, gõ lại `whoami`, `id`, `groups`. Tạo nhanh 1 file rồi `chmod 600` và `chmod 644`, đọc lại `ls -l` đến khi giải mã được không cần nhìn bảng (nhớ: dấu `@` trên macOS chỉ là thuộc tính mở rộng, bỏ qua). Đọc lại “Thẻ ghi nhớ” một lượt.
- **Sau 3 ngày:** Không nhìn sách, tự làm lại **toàn bộ lab B** (script `chao.sh`: tạo → chạy thất bại → `chmod u+x` → chạy được). Nhắm 755/644/600 từ trí nhớ.
- **Sau 1 tuần:** Giải lại tất cả “Tự kiểm tra” + “Bài tập mở rộng” không xem đáp án. Tự giải thích thành lời cho người khác (hoặc cho chính mình): “*Vì sao chmod 777 nguy hiểm?*” và “*sudo khác su ở đâu?*”. Nếu giải thích trôi chảy = đã thuộc.

14 Tóm tắt 1 trang

- **Đa người dùng → cần phân quyền.** Mỗi file có **owner / group / others** + tài khoản tối cao **root**.
- **Tôi là ai:** `whoami` (tên), `id` (uid/gid/nhóm), `groups`. macOS: nhóm chính của phiên thường là `staff`, uid bắt đầu ~501.
- **Đọc `ls -l`** chuỗi 10 ký tự: `[loại][owner rwx][group rwx][others rwx]`. `-`=file, `d`=thư mục, `l`=link (còn `c/b/p/s` ít gặp). Trên macOS có thể thấy thêm `@` (extended attributes) hoặc `+` (ACL) sau 10 ký tự — không phải quyền.
- **Quyền:** `r=4` (đọc), `w=2` (ghi), `x=1` (chạy file / vào thư mục).
- **`chmod` ký hiệu:** `u/g/o/a + +/=/- = + r/w/x`. VD `chmod u+x`, `chmod go-w`. Nhớ: `+` cộng dồn, `=` đặt tuyệt đối (xóa quyền không liệt kê).
- **`chmod` số (octal, mỗi vai 0-7):** 3 chữ số = owner/group/others, mỗi số là tổng `4+2+1`.
 - `755 = rwxr-xr-x` (script/thư mục cần truy cập) · `644 = rw-r--r--` (file thường)
 - `600 = rw-----` (riêng tư/khóa). Thư mục/file riêng tư cần nhắc 700/750.
- **`chown/chgrp`** đổi chủ/nhóm — **khác `chmod`** (đổi quyền). Đổi chủ sang user khác **luôn cần `sudo`**; chỉ để nhận diện, không chạy trong lab.
- **`sudo`** = mượn quyền quản trị cho **một lệnh**; macOS hỏi mật khẩu đăng nhập của **chính bạn** (gõ không hiện ký tự); chỉ gõ khi **chính bạn** chủ động gõ `sudo`. Khác `su` (chuyển hẳn user). macOS tắt đăng nhập trực tiếp bằng `root` nhưng `sudo` vẫn chạy với tư cách `root`. Dùng dè dặt; tránh `sudo rm -r` với `/` hoặc `~`.
- **Senior:** least privilege là gốc; `chmod 777` (nhất là `-R 777`) nguy hiểm; khóa SSH bắt buộc 600; `sudo` có log để audit; đây là nền của IAM cloud sau này.

15 Bài tập mở rộng (có đáp án)

Chuẩn bị sân tập

Làm trong sân tập an toàn. Trước khi bắt đầu: `mkdir -p ~/linux-lab/ch5-bt && cd ~/linux-lab/ch5-bt`. Xong nhớ dọn: `cd ~&& rm -r ~/linux-lab/ch5-bt`.

BT1. Tạo file `note.txt` rồi đặt quyền sao cho **chủ đọc+ghi, group chỉ đọc, others không có gì**. Viết lệnh (cả 2 cách: số và ký hiệu) và cho biết chuỗi `ls -l` mong đợi.

BT2. Có file `run.sh` đang là `-rw-r--r--`. Hãy làm cho **mọi người đều chạy được** nhưng **chỉ chủ sửa được**. Viết lệnh số, và lệnh ký hiệu tương đương.

BT3. Giải mã bằng lời: `drwxr-x---`. Đây là gì, ai làm được gì?

BT4. Vì sao đặt `chmod 777 ~` (cả thư mục nhà) là ý tưởng tệ? Nêu 2 lý do.

BT5. Bạn gặp lỗi `ssh` từ chối khóa riêng và than phiền “permissions too open”. Lệnh nào sửa, và vì sao?

Đáp án Bài tập mở rộng

BT1.

```
echo "ghi chu" > note.txt
chmod 640 note.txt          # cách số
# hoặc cách ký hiệu tương đương:
chmod u=rw,g=r,o= note.txt
ls -l note.txt
```

Chuỗi mong đợi: `-rw-r-----` (owner `rw-`, group `r--`, others `---` = số 640). (Trên macOS có thể có thêm dấu `@`: `-rw-r-----@` — bình thường.)

BT2.

```
chmod 755 run.sh            # cách số -> rwxr-xr-x
# hoặc ký hiệu, DAT TUYET DOI cho chac chan:
chmod u=rwx,go=rx run.sh
ls -l run.sh
```

Kết quả: `-rwxr-xr-x`. Mọi người chạy/đọc được, chỉ owner có `w` (sửa).

Lưu ý sự phạm quan trọng

Đề bài cho file đang là `-rw-r--r--`. Trong trường hợp đó, `chmod a+x run.sh` cũng *ra đúng* `-rwxr-xr-x` — nhưng đó chỉ là **may mắn nhờ trạng thái ban đầu**. `+x` chỉ **thêm** `x`, KHÔNG đụng tới `w`; nếu file ban đầu lỡ có `w` cho group/others (vd vừa bị `chmod 666`), thì `a+x` sẽ KHÔNG khóa được quyền sửa của người khác → vi phạm yêu cầu “chỉ chủ sửa được”. Vì vậy, khi cần **đảm bảo** kết quả, hãy dùng cách số (755) hoặc ký hiệu **đặt tuyệt đối** (`u=rwx,go=rx`), đừng dựa vào `+x`.

BT3. `drwxr-x---`:

- Ký tự đầu `d` → **thư mục**.

- **rwX** (owner): chủ đọc nội dung, ghi/tạo/xóa file bên trong, và **cd** vào được.
- **r-x** (group): thành viên nhóm đọc danh sách và **cd** vào được, **không** tạo/xóa file.
- **---** (others): người ngoài **không thấy, không vào** được gì.

BT4. `chmod 777 ~` tệ vì:

1. **Bảo mật:** mọi người dùng/tiến trình khác đều **ghi** được vào thư mục nhà của bạn → có thể thay/cắm file độc, xóa dữ liệu.
2. **Phá công cụ dựa trên quyền:** nhiều chương trình (vd `ssh`) **từ chối hoạt động** khi thấy thư mục/khóa quyền quá mở, nên thay vì “đỡ lỗi” bạn lại **gây thêm lỗi**; đồng thời vi phạm least privilege.

BT5.

```
chmod 600 ~/.ssh/id_ed25519 # (ten khoa co the khac, vd id_rsa)
```

Vì khóa riêng SSH **bắt buộc** chỉ owner đọc/ghi (`rw-----` = 600). Nếu group/others đọc được, `ssh` coi là không an toàn (người khác có thể trộm khóa) nên **từ chối dùng**. Đặt 600 khôi phục đúng quyền tối thiểu, `ssh` chấp nhận lại.

Đáp án các mục “Tự kiểm tra”

Đáp án 5.1

1. Owner (chủ), group (nhóm), others (người khác).
2. root là superuser — quyền tối cao, bỏ qua mọi rào quyền, làm được mọi thứ.
3. Vì hệ điều hành vẫn chạy nhiều tài khoản/tiến trình hệ thống cùng lúc; phân quyền cô lập chúng để bảo mật và tránh phá nhau.

Đáp án 5.2

1. `whoami`.
2. `uid` định danh **người dùng**; `gid` định danh **nhóm chính** (primary group) của phiên làm việc.
3. `staff`.

Đáp án 5.3

1. owner: đọc + ghi (`rw-`); others: chỉ đọc (`r--`).
2. `r=4, w=2, x=1`.
3. Liệt kê được tên file bên trong (nhờ `r`), nhưng **không cd vào được** và không truy cập nội dung (thiếu `x`).

Đáp án 5.4

1. Vì file `chao.sh` chưa có quyền `x` (đang `-rw-r--r--`), hệ thống không cho chạy.

2. Thêm quyền thực thi (x) cho owner → thành **-rwxr--r--**.
3. **./** = “file nằm ngay trong thư mục hiện tại” (chỉ rõ vị trí để shell tìm và chạy).

Đáp án 5.5

1. Bỏ quyền **ghi (w)** của **group và others** trên file abc.
2. **5=r-x** (đọc+chạy); **6=rw-** (đọc+ghi); **7=rwx** (đọc+ghi+chạy).
3. **chmod 600** → **rw-----**; phù hợp **file riêng tư** như khóa SSH (chỉ chủ đọc/ghi).

Đáp án 5.6

1. Không thấy gì (group và others là ---); chỉ owner đọc/ghi được.
2. Ở các chữ x: 755 có x cho cả ba vai (**rwxr-xr-x**), 644 không có x nào (**rw-r--r--**).
3. **rm -r** xóa **đệ quy** cả thư mục và mọi thứ bên trong; **rm** thường chỉ xóa **một file** (không xóa được thư mục có nội dung).

Đáp án 5.7

1. *superuser do*; dùng để chạy **một lệnh** với quyền quản trị (root), vd sửa file hệ thống/cài đặt.
2. Mật khẩu **đăng nhập của chính bạn** (với điều kiện tài khoản của bạn thuộc nhóm admin) trên macOS; màn hình ẩn hoàn toàn ký tự là **cố ý** — để người nhìn qua vai (shoulder surfing) không đọc/đoán được mật khẩu; cứ gõ và Enter.
3. **sudo** mượn quyền cho **đúng một lệnh** rồi trả lại (hẹp, có log); **su chuyển hẳn** sang user khác và ở lại cho tới khi **exit**.

Chuẩn bị cho Chương 6

Bạn đã kiểm soát được “ai làm gì” với file. Chương sau ta sẽ bước vào **tiến trình & quản lý chương trình đang chạy** (xem, tìm, dừng tiến trình) — nơi khái niệm “chủ sở hữu” và quyền lại tiếp tục phát huy. Hẹn gặp lại!